

Prompt Engineering

Talking to large language models — clearly, deliberately, and effectively.

Sourav Karmakar

souravkarmakar29@gmail.com

What we'll cover



01

Foundations



How language models actually work — tokens, prediction, temperature & sampling.

02

Core Techniques



The everyday toolkit: zero/few-shot, clear instructions, roles, formatting, chain-of-thought.

03

Structuring & Workflows



System prompts, templates, task-specific prompting, chaining & context management.

04

Production: Cost, Quality & Safety



Token/cost engineering, evaluation, ethics, and the pitfalls to avoid.

What is prompt engineering?



The skill of designing the input you give a model so it reliably produces the output you want.

It is mostly about communication — not code:

- Say exactly what you want, and in what form
- Give the model the context it needs
- Test, observe what fails, and refine

Why it matters



Same model, better prompt

Often the difference between useless and excellent output.



Cheaper than alternatives

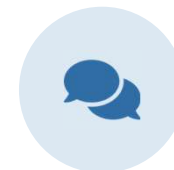
No training, no data pipeline — just words.



Transferable skill

Works across ChatGPT, Claude, Gemini and APIs.

What is a prompt?



A prompt is simply the text you send to a model. The model reads it and continues it with a response.



Key idea: the model doesn't "understand" the way people do. It predicts what text should come next, based on patterns learned from huge amounts of writing. A better prompt simply steers that prediction toward what you actually want.

PART

01



Foundations: How Models Work

A light, intuition-first look under the hood — just enough to make every later choice make sense.

HOW MODELS WORK

Models predict the next token

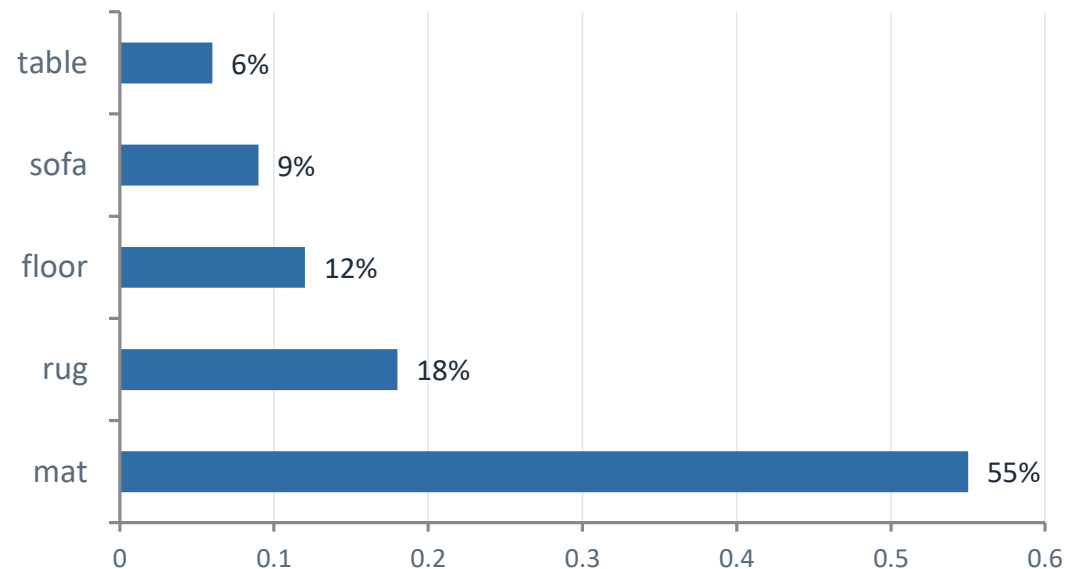


At each step the model looks at everything so far and assigns a probability to every possible next token. It picks one, adds it, and repeats.

PROMPT SO FAR

"The cat sat on the ____"

The model now ranks every candidate for the blank by probability →



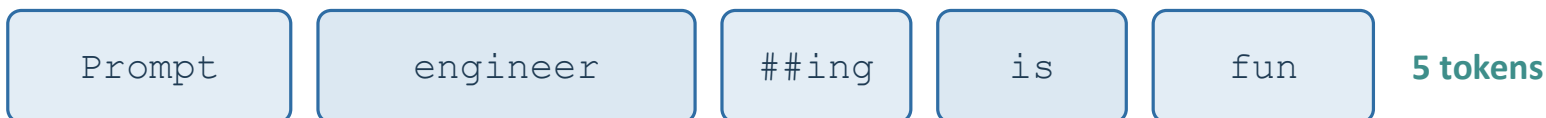
HOW MODELS WORK

Tokens & the context window



Models don't read words or letters — they read tokens: common chunks of text. Roughly 1 token $\approx$ 4 characters $\approx$ $\frac{3}{4}$ of a word in English.

“Prompt engineering is fun” breaks into:



THE CONTEXT WINDOW

The model can only “see” a limited number of tokens at once — prompt + response combined. Go over the limit and the earliest text is dropped and forgotten.

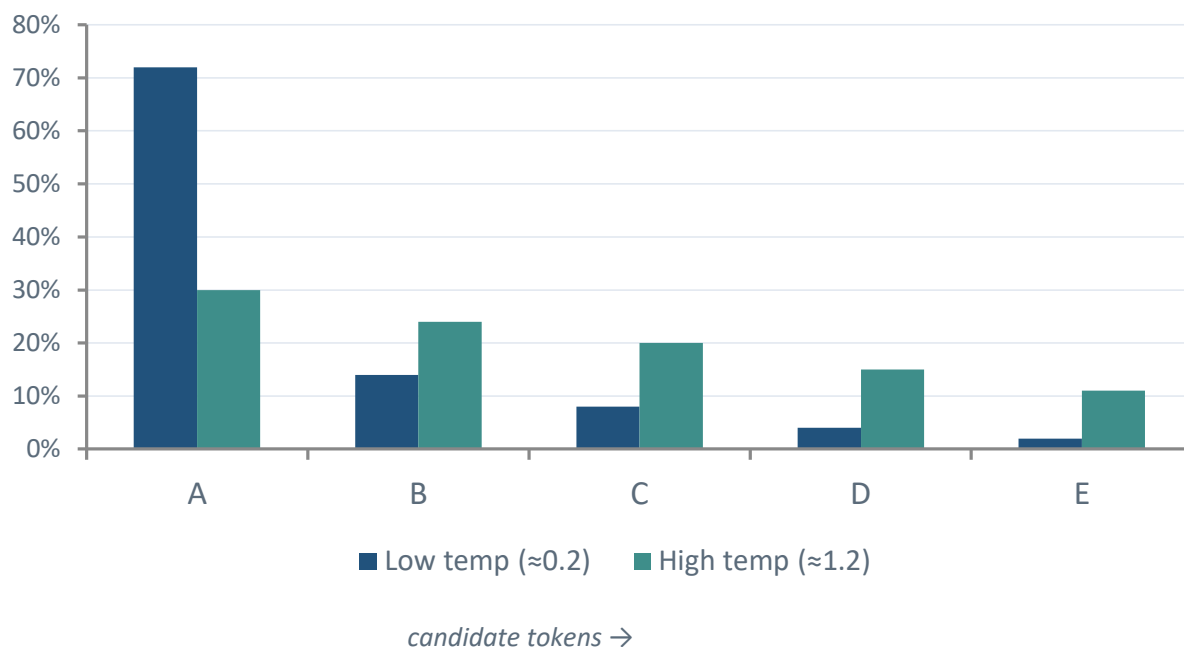


Prompt + reply share one budget

Temperature: creativity vs consistency



Temperature reshapes the probabilities before a token is chosen. Low = play it safe (pick likely tokens); high = take risks (flatten the odds).



A little math

Each score z is divided by T before softmax:

$$p \propto \exp(z / T)$$

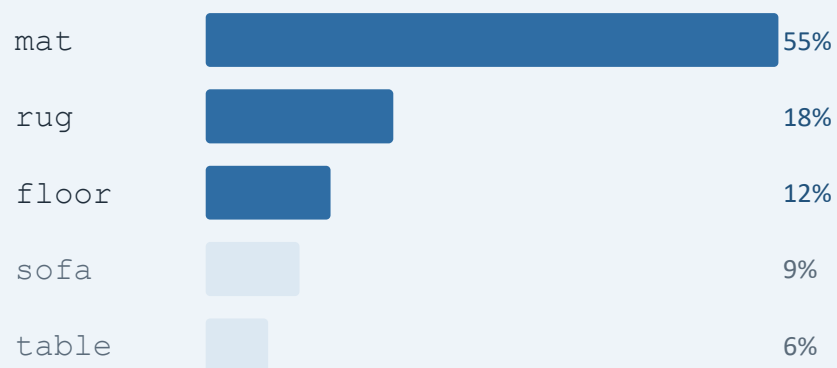
- $T \rightarrow 0$: nearly always the top token
- $T = 1$: the model's raw odds
- $T > 1$: flatter, more surprising

Sampling: top-k and top-p



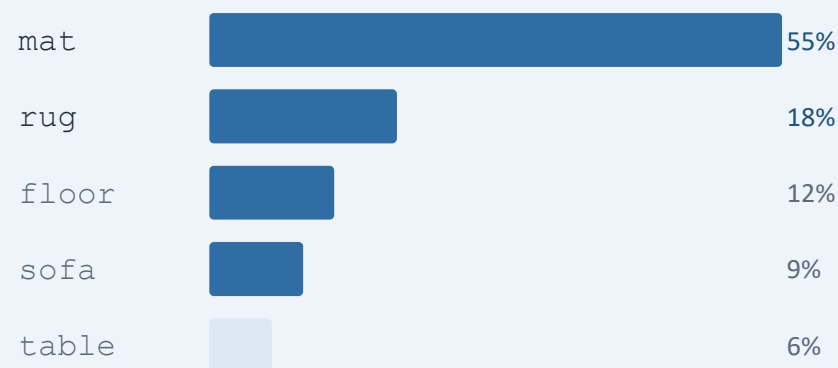
After temperature, the model still must pick one token. Sampling decides which candidates are even allowed in the running.

top-k (k = 3)



Keep the k most likely tokens, drop the rest, then sample.

top-p (p = 0.90)



Keep the smallest set whose probabilities sum to $\geq p$ (here $55\%+18\%+12\%\approx 85\%$...add 'sofa' $\rightarrow 94\%$).

PART

02



Core Prompting Techniques

The everyday toolkit — the handful of moves you'll reach for in almost every prompt you write.

CORE TECHNIQUES

Anatomy of a good prompt



Most strong prompts contain some mix of these building blocks. You won't always need all five — but naming them helps you spot what's missing.

1

Role

Who the model should act as

2

Context

Background it needs to know

3

Instruction

The actual task to perform

4

Input

The data to work on

5

Output format

How the answer should look

PUTTING IT TOGETHER

You are a careful copy-editor (role). I'm writing a resume for a data-analyst job (context). Fix grammar and make it punchier (instruction). Here is the text: "... (input). Return the edited version, then 3 bullet notes on what you changed (format).

Fine-tuning vs prompting



Two ways to make a model behave how you want. For almost everything a beginner does, prompting is the right tool.



Prompting

Effort	Write words. Minutes.
Cost	Cost of tokens as per the model.
Data	None required.
Best for	Almost all everyday tasks.



Fine-tuning

Effort	Train on examples. Days+.
Cost	Compute cost + expertise.
Data	100s–1000s of examples.
Best for	Narrow, repeated, high-volume tasks.

Rule of thumb: try prompting first. Only consider fine-tuning when prompting plateaus on a specific, repetitive task at scale.

Zero-shot prompting



Zero-shot = you ask the model to do the task directly, giving no examples of the answer. You rely on what it already learned in training.

PROMPT

Classify the sentiment of this review as positive, negative, or neutral:

"The battery dies in two hours. Very disappointing."

RESPONSE

Negative

When it shines

- Common, well-understood tasks
- Quick drafts and one-offs
- When you want minimal setup

Why it works

Modern models have seen millions of similar tasks, so a clear instruction is often enough on its own.

Where zero-shot falls short



With no examples to anchor it, zero-shot can misread what you want. Watch for these failure modes.



Ambiguous format

It guesses the output shape — you may get prose when you wanted a list or JSON.



Niche or custom tasks

If your task isn't 'standard', the model has no pattern to fall back on.



Subjective labels

Your definition of 'urgent' or 'on-brand' may differ from the model's.



Confident hallucination

It may invent facts rather than admit uncertainty.

The usual fix: show, don't just tell — add a few examples. That's few-shot prompting (next).

Few-shot prompting



Few-shot = include a few worked examples in the prompt. The model copies the pattern — the format, style, and labels you demonstrate.

PROMPT (WITH 2 EXAMPLES)

Extract the company and amount.

Text: "Paid \$40 to Netflix." → {company: Netflix, amount: 40}

Text: "Acme charged me \$9." → {company: Acme, amount: 9}

Text: "Spent \$120 at Uber." →

RESPONSE

{company: Uber, amount: 120}

Tips

- 2–5 examples is usually plenty
- Make examples cover the tricky cases
- Keep the format identical across all
- More examples = more tokens & cost
- Great for custom labels & strict output

Writing clear instructions



The single biggest lever for beginners. Vague prompts get vague answers. Be specific about task, scope, audience, and format.

Vague

`"Tell me about climate change."`

*→ Could be 3 lines or 3 pages, any reading level, any angle.
You'll spend turns correcting it.*

Specific

`"In 5 bullet points for a high-schooler, explain the main human causes of climate change. Keep it under 120 words."`

*→ Scope, audience, format, and length are all pinned down.
One clear answer.*

CORE TECHNIQUES

Roles & personas



Telling the model who to be sets vocabulary, depth, and tone in one short phrase — a fast way to shape the whole response.

"Act as a patient tutor..."

Simple, encouraging, step-by-step

"You are a senior lawyer..."

Precise, cautious, formal

"Reply like a witty friend..."

Casual, playful, brief

SAME QUESTION, PERSONA SET

"You are a friendly chemistry teacher for beginners. What is an atom?"

RESPONSE

Great question! Picture a tiny LEGO brick that everything is built from...

Controlling the output format



Don't accept whatever shape the model returns — ask for the exact structure you need. This makes answers usable and, with APIs, machine-readable.

PROMPT

List 3 EVs and their range. Return ONLY valid JSON: an array of objects with keys "model" and "range_miles".

RESPONSE

```
[ {"model": "Model 3", "range_miles": 272},  
  {"model": "Ioniq 6", "range_miles": 361},  
  {"model": "Leaf", "range_miles": 212} ]
```

Formats you can ask for

- Bullet lists or numbered steps
- Markdown tables
- JSON / CSV for code
- A fixed template you provide
- Length limits & tone

Iterative prompting



A great prompt is rarely written in one go. Treat it like editing: draft, look at what's wrong, adjust one thing, repeat.



repeat until it's right

Chain-of-thought prompting



For reasoning, math, or multi-step problems, ask the model to think step by step before answering. Showing its work improves accuracy.

DIRECT (OFTEN WRONG)

Q: A bat and ball cost \$1.10. The bat costs \$1 more than the ball. Ball = ?
A: \$0.10

RESULT

Wrong — jumped to the intuitive answer.

WITH “LET’S THINK STEP BY STEP”

Ball = x , bat = $x+1$. $x+(x+1)=1.10 \rightarrow 2x=0.10$
 $\rightarrow x=0.05$.

RESULT

Correct — \$0.05. Reasoning catches the trap.

Add “Explain your reasoning step by step, then give the final answer.” — trades a few extra tokens for big accuracy gains on hard problems.

PART

03



Structuring & Workflows

Moving from single prompts to organized, reusable, multi-step systems.

System prompt vs user prompt



Many tools and all APIs split the input in two: a system prompt that sets persistent rules, and user prompts for each request.



System prompt

Set once · applies to the whole conversation

`"You are a support agent for ABCD Bank. Be polite, never give legal advice, and keep replies under 4 sentences."`



User prompt

Changes every turn · the actual request

`"I lost my card, what do I do?"`

`"Can you also freeze my account?"`

Prompt templates



Once a prompt works, turn it into a reusable template with {placeholders}. Fill in the blanks each time instead of rewriting from scratch.

TEMPLATE

```
Summarize the {document} below for a  
{audience}.  
Use {n} bullet points. Tone: {tone}.  
  
---  
{document}
```



FILLED IN

```
Summarize the earnings call below  
for a busy executive. Use 3 bullet  
points. Tone: neutral.  
  
--- "In Q3 we..."
```

Benefits: consistency across uses · easy to share with a team · change once, improve everywhere.

Task-specific prompting



Different jobs reward different prompt shapes. A few common patterns worth memorizing:



Summarize

"Summarize in 3 bullets, $\leq$ 60 words."



Classify

"Label as spam / not spam. One word."



Translate

"Translate to French, keep the tone."



Extract

"Pull all dates as YYYY-MM-DD, JSON."



Generate

"Write 5 subject-line options."



Rewrite

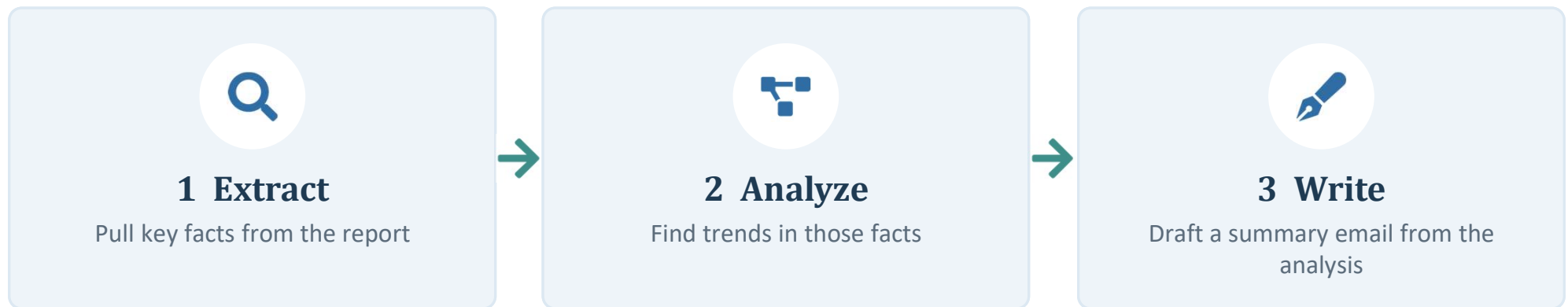
"Make this paragraph clearer & shorter."

STRUCTURING

Prompt chaining

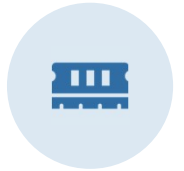


For complex jobs, don't cram everything into one prompt. Break the task into steps and feed each output into the next.



Why chain? each step is simpler and more reliable, you can inspect intermediate results, and you can swap or fix one step without redoing the rest.

Context management



Remember the context window has a fixed budget. Managing what goes in it is its own skill: include what helps, leave out the noise.



Put in

- The specific data the task needs
- Relevant examples (few-shot)
- Clear instructions & output format
- Only the parts of long docs that matter



Leave out

- Irrelevant history & chit-chat
- Whole documents “just in case”
- Repeated or contradictory instructions
- Stale context from earlier topics

PART

04



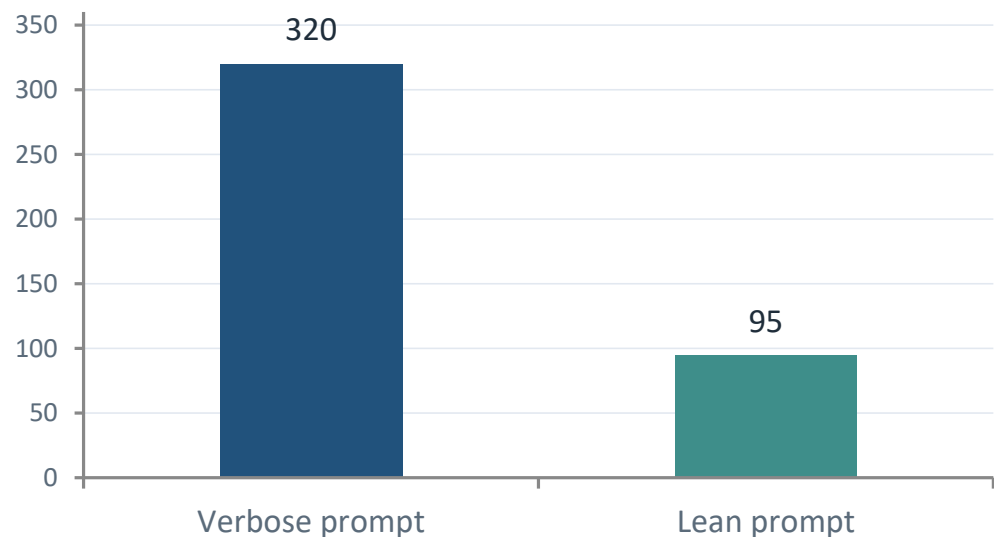
Production: Cost, Quality & Safety

What turns a clever demo into something you can actually rely on and ship.

Token optimization & cost



APIs charge per token — both what you send and what you get back. Leaner prompts are cheaper and often faster, with no loss of quality.



Same task, same answer — ~70% fewer tokens.

Ways to trim

- Cut filler & repeated instructions
- Cap the answer length you ask for
- Send only the relevant excerpt
- Reuse a tight template
- Use a smaller model for easy tasks

Prompt caching



Reusing an unchanging chunk of a prompt across calls so the model doesn't reprocess — and you don't re-pay for — the same tokens every time.

WITHOUT CACHING

Fixed prefix: system + context + examples

new Q

Reprocessed in full on every call — you pay for the whole prompt each time.

WITH CACHING

Cached prefix — reused, not re-billed

new Q

On later calls the prefix is served from cache — big cost & latency savings; you process only the new part.

When it helps

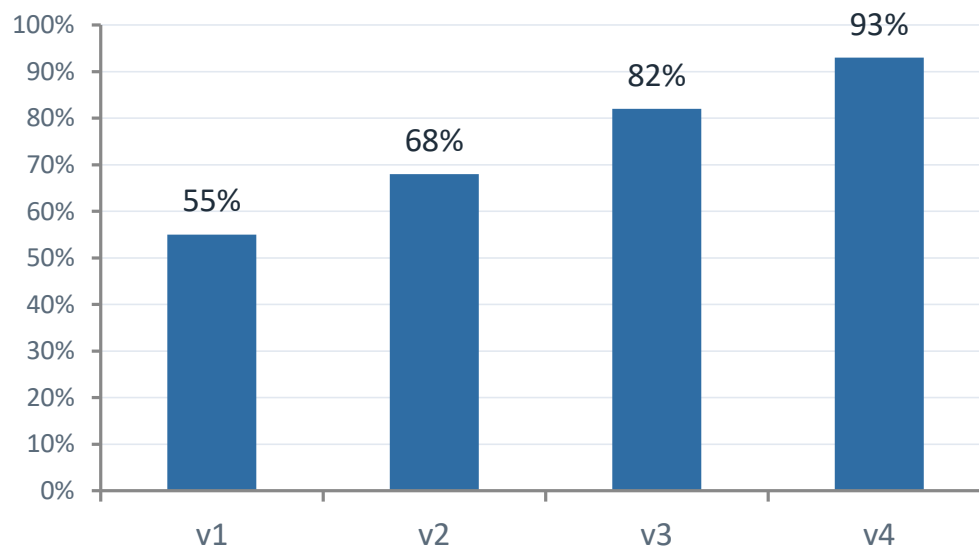
- Long, reused system prompts
- A big document you query repeatedly
- Many-shot examples reused across calls
- Long multi-turn conversations

Rule: cache the unchanging part, keep it at the very start, and make it byte-for-byte identical each call.

Evaluating & testing prompts



“It looked good once” isn't evidence. To trust a prompt, test it on many cases and measure how often it succeeds.



Track pass rate as you iterate on prompt versions.

How to test

- Build a small set of test inputs
- Define what “correct” means up front
- Score each output: pass / fail
- Compare prompt versions on the same set
- Re-test after every change

Ethics & responsible use



Prompting is powerful, and that comes with responsibility. Keep these front of mind in anything you build or share.



Verify facts

Models can state false things fluently. Check anything that matters.



Watch for bias

Outputs can reflect biases in training data. Review sensitive uses.



Protect privacy

Don't paste personal or confidential data into prompts carelessly.



Be transparent

Disclose AI-generated content; don't pass it off as human when it counts.

Common pitfalls to avoid



The mistakes beginners hit most often — and the quick fix for each.



Being too vague



Specify task, audience, format, length.



Asking for too much at once



Split into steps or chain prompts.



Trusting output blindly



Verify facts; test on multiple cases.



Ignoring output format



State the exact structure you need.



Overstuffing context



Include only what's relevant.



Giving up after one try



Iterate — refine one thing at a time.

Key takeaways



If you remember only these, you're already an effective prompter:

- ✓ Models predict the next token — prompts steer that prediction.
- ✓ Be specific: task, context, audience, and output format.
- ✓ Show, don't just tell — examples (few-shot) beat description.
- ✓ For reasoning, ask the model to think step by step.
- ✓ Iterate: draft → test → refine one thing → repeat.
- ✓ Verify, evaluate, and use AI responsibly.

KEEP LEARNING

Where to go next



Practice daily

Apply prompting to one real task at work or study each day.



Read the model docs

Provider prompting guides (OpenAI, Anthropic, Google) are free and excellent.



Keep a prompt journal

Save prompts that work and templates you can reuse.



Test & compare

Try the same task across models and settings to build intuition.

Thank You